

Space Weather Radio-Risk Monitor

A Kafka alert for amateur-radio operators when planetary Kp crosses storm level

Noam Fein & Niki Naderzad · MSDS 682 Data Stream Processing · August 13, 2026

The problem, the user, the result

Presentation rubric 1 — problem, user, useful result, why it matters

- **User:** an amateur-radio operator who relies on HF propagation.
- **Problem:** geomagnetic storms degrade HF; NOAA's Kp estimate updates every minute, but nobody wants to watch a feed.
- **Rule:** alert when the rolling 15-minute max Kp crosses from **below 6** to ≥ 6 (G2 storm — NOAA: HF can fade at higher latitudes). Rearm only after it falls below 6.

Result: a validated Kafka stream turns 9 replayed observations into **exactly 2 machine-readable alerts** — no spam while conditions stay elevated, no AI in the decision.

NOAA source → one validated event

Presentation rubric 2 — source, Kafka event, key fields, representative record

- **Source:** NOAA SWPC
planetary_k_index_1m.json — public, no key.
Values are *estimated, preliminary, global*.
- Raw `estimated_kp` → canonical `kp_value`; every event passes one Pydantic contract (UTC times, Kp in 0–9, documented provenance only).
- **Topic** `kp_observations`, **constant key** `planetary_kp`: one global sequence stays ordered. Dedup by `time_tag`.
- Quiet feed (checked Aug 11 + 12: max Kp < 3.5) → a **labeled synthetic fixture** is the required alert demo.

```
{  
  "time_tag":    "2026-08-11T00:10:00Z",  
  "kp_value":    6.3,  
  "source":      "synthetic://kp-threshold-fixture",  
  "ingested_at": "2026-08-11T00:10:05Z"  
}
```

The representative event — the record that produces alert #1. Synthetic provenance is explicit and validator-enforced.

Implemented architecture

Presentation rubric 3 — architecture, concrete example, design reason



- **Why this design:** one global Kp series → one topic, one partition, constant key preserves order. Kafka decouples validated producer from consumer and makes duplicates observable.
- Demo resets the broker every run — **no stale offsets, fully reproducible.**

The replay, row by row

Actual metrics.csv from the verified run — one row per consumed Kafka message

time_tag (UTC)	Kp	15-min max	label	alert?	status
00:00	4.0	4.0	normal	–	accepted
00:05	5.0	5.0	normal	–	accepted
00:10	6.3	6.3	elevated	ALERT #1	accepted
00:10 (again)	6.3	6.3	elevated	–	duplicate_skipped
00:20	5.5	6.3	elevated	– (still elevated)	accepted
00:25	5.2	6.3	elevated	–	accepted
00:26	5.0	5.5	normal	– (rearmed)	accepted
00:27	7.0	7.0	elevated	ALERT #2	accepted
00:30	6.7	7.0	elevated	–	accepted

One table shows everything: the crossing, the skipped duplicate, no alert-spam while elevated, window decay rearming the rule, and a second crossing.

One command, verified evidence

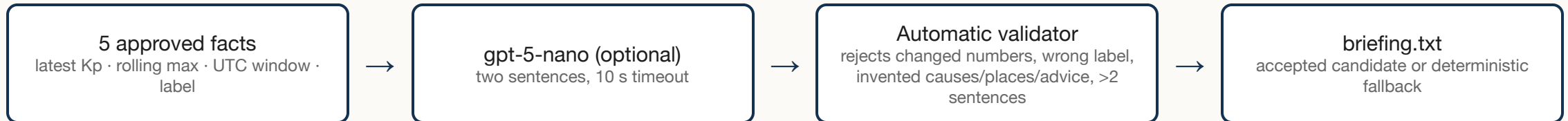
`./run_demo.sh` — reset broker → replay through real Kafka → 4 artifacts → verify → test

- **141 tests passed** + 3 Kafka integration tests on a clean broker.
- Artifacts checked **field-by-field** against a committed expected-results oracle.
- Three consecutive runs identical; broken run exits nonzero.
- Passes with **no network beyond localhost and no API key** (`env -u OPENAI_API_KEY`).

```
$ ./run_demo.sh
Processed 9 valid event(s); skipped 0 malformed;
emitted 2 alert(s)
Artifacts match the labeled fixture: 2 alert(s),
9 metrics row(s), briefing present,
evaluation 7/7 assertions passed.
141 passed, 3 skipped
Demo complete.
```

AI writes sentences, never decisions

Bounded briefing + automatic verification + deterministic fallback



- Six committed evaluation cases (correct / wrong number / wrong label / unsupported detail / too long / API down)
 - **all decided as expected, 7/7 assertions**, no network needed to re-check.
- The AI can never trigger, suppress, or relabel an alert. Rejection is recorded, never silently repaired.

Lesson, limitation, next step

Presentation rubric 4 — evidence-based lesson and a realistic next step

- **Lesson:** deterministic replay beat live data for proving stream logic — a quiet feed can never show a threshold crossing, but a labeled fixture shows crossing, dedup, and rearm every single run.
- **Limitation:** this reports a recent estimated *global* Kp condition — not a forecast, not a station-level propagation model; the validator checks facts, not eloquence.
- **Next step:** the deferred 60-second live NOAA poller — the contract, topic, and consumer already accept normalized NOAA records unchanged.

Everything you saw is reproducible from the ZIP with one command and no credentials.

Backup: saved terminal evidence

If Docker, Wi-Fi, NOAA, or OpenAI fails on stage, this slide is the demo

```
$ ./run_demo.sh
==> Resetting the local Kafka broker (a fresh broker prevents stale topics and offsets)
==> Starting the finite Kafka consumer in the background
==> Publishing the nine-message deterministic replay through Kafka
Published 9 event(s) · Topic=kp_observations key=planetary_kp partitions=[0]
Processed 9 valid event(s); skipped 0 malformed; emitted 2 alert(s)
==> Writing the deterministic briefing (no API key or network required)
Elevated radio-risk conditions were detected with a latest Kp of 7 and a rolling
15-minute maximum of 7. The UTC window ran from 2026-08-11T00:12:00Z to
2026-08-11T00:27:00Z, and the rule-based risk label was elevated.
==> Writing deterministic evaluation evidence
Evaluation: 7/7 assertions passed
==> Verifying artifacts against data/fixtures/replay_expected.json
Artifacts match the labeled fixture: 2 alert(s), 9 metrics row(s),
briefing present, evaluation 7/7 assertions passed.
==> Running the automated test suite
141 passed, 3 skipped
Demo complete. Artifacts: outputs/alert.json · outputs/metrics.csv ·
outputs/briefing.txt · evaluation/evaluation.json
```